

thermochemical_justification

June 1, 2026

1 KNSB Thermochemical Justification

1.1 1. Setting up the environment

I'll be using a powerful tool known as [PyProPEP](#) for this task. It's a Python interface to [CProPEP](#) (*an improvement on ForTran ProPEP*). It calculates everything accurately and efficiently. Also ensure you've set up Jupyter on your machine.

First, install the libraries:

```
[ ]: %pip install --no-cache-dir -r requirements.txt
```

```
Collecting ipykernel (from -r requirements.txt (line 1))
  Downloading ipykernel-7.2.0-py3-none-any.whl.metadata (4.5 kB)
Collecting numpy (from -r requirements.txt (line 2))
  Downloading
numpy-2.4.6-cp311-cp311-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata
(6.6 kB)
Collecting pandas (from -r requirements.txt (line 3))
  Downloading pandas-3.0.3-cp311-cp311-
manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.metadata (79 kB)
Collecting matplotlib (from -r requirements.txt (line 4))
  Downloading matplotlib-3.10.9-cp311-cp311-
manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (52 kB)
Collecting scipy (from -r requirements.txt (line 5))
  Downloading scipy-1.17.1-cp311-cp311-
manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (62 kB)
Collecting pypropep (from -r requirements.txt (line 6))
  Downloading pypropep-0.1.4.tar.gz (346 kB)
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Installing backend dependencies ... done
  Preparing metadata (pyproject.toml) ... done
Collecting sympy (from -r requirements.txt (line 7))
  Downloading sympy-1.14.0-py3-none-any.whl.metadata (12 kB)
Collecting Fraction (from -r requirements.txt (line 8))
  Downloading Fraction-2.2.0-py3-none-any.whl.metadata (2.8 kB)
Collecting comm>=0.1.1 (from ipykernel->-r requirements.txt (line 1))
  Downloading comm-0.2.3-py3-none-any.whl.metadata (3.7 kB)
```

```

Collecting debugpy>=1.6.5 (from ipykernel->-r requirements.txt (line 1))
  Downloading debugpy-1.8.20-cp311-cp311-manylinux_2_34_x86_64.whl.metadata (1.4
kB)
Collecting ipython>=7.23.1 (from ipykernel->-r requirements.txt (line 1))
  Downloading ipython-9.14.0-py3-none-any.whl.metadata (4.7 kB)
Collecting jupyter-client>=8.8.0 (from ipykernel->-r requirements.txt (line 1))
  Downloading jupyter_client-8.8.0-py3-none-any.whl.metadata (8.4 kB)
Collecting jupyter-core!=6.0.*,>=5.1 (from ipykernel->-r requirements.txt (line
1))
  Downloading jupyter_core-5.9.1-py3-none-any.whl.metadata (1.5 kB)
Collecting matplotlib-inline>=0.1 (from ipykernel->-r requirements.txt (line 1))
  Downloading matplotlib_inline-0.2.2-py3-none-any.whl.metadata (2.4 kB)
Collecting nest-asyncio>=1.4 (from ipykernel->-r requirements.txt (line 1))
  Downloading nest_asyncio-1.6.0-py3-none-any.whl.metadata (2.8 kB)
Collecting packaging>=22 (from ipykernel->-r requirements.txt (line 1))
  Downloading packaging-26.2-py3-none-any.whl.metadata (3.5 kB)
Collecting psutil>=5.7 (from ipykernel->-r requirements.txt (line 1))
  Downloading psutil-7.2.2-cp36-abi3-
manylinux2010_x86_64.manylinux_2_12_x86_64.manylinux_2_28_x86_64.whl.metadata
(22 kB)
Collecting pyzmq>=25 (from ipykernel->-r requirements.txt (line 1))
  Downloading pyzmq-27.1.0-cp311-cp311-
manylinux_2_26_x86_64.manylinux_2_28_x86_64.whl.metadata (6.0 kB)
Collecting tornado>=6.4.1 (from ipykernel->-r requirements.txt (line 1))
  Downloading tornado-6.5.6-cp39-abi3-
manylinux1_x86_64.manylinux_2_28_x86_64.manylinux_2_5_x86_64.whl.metadata (2.8
kB)
Collecting traitlets>=5.4.0 (from ipykernel->-r requirements.txt (line 1))
  Downloading traitlets-5.15.0-py3-none-any.whl.metadata (10 kB)
Collecting python-dateutil>=2.8.2 (from pandas->-r requirements.txt (line 3))
  Downloading python_dateutil-2.9.0.post0-py2.py3-none-any.whl.metadata (8.4 kB)
Collecting contourpy>=1.0.1 (from matplotlib->-r requirements.txt (line 4))
  Downloading contourpy-1.3.3-cp311-cp311-
manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (5.5 kB)
Collecting cycycler>=0.10 (from matplotlib->-r requirements.txt (line 4))
  Downloading cycycler-0.12.1-py3-none-any.whl.metadata (3.8 kB)
Collecting fonttools>=4.22.0 (from matplotlib->-r requirements.txt (line 4))
  Downloading fonttools-4.63.0-cp311-cp311-
manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (118 kB)
Collecting kiwisolver>=1.3.1 (from matplotlib->-r requirements.txt (line 4))
  Downloading kiwisolver-1.5.0-cp311-cp311-
manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (5.1 kB)
Collecting pillow>=8 (from matplotlib->-r requirements.txt (line 4))
  Downloading pillow-12.2.0-cp311-cp311-
manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (8.8 kB)
Collecting pyparsing>=3 (from matplotlib->-r requirements.txt (line 4))
  Downloading pyparsing-3.3.2-py3-none-any.whl.metadata (5.8 kB)
Collecting cffi>=1.0.0 (from pypropep->-r requirements.txt (line 6))

```

Downloading
 cffi-2.0.0-cp311-cp311-manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata
 (2.6 kB)
 Collecting attrdict (from pyprosep->-r requirements.txt (line 6))
 Downloading attrdict-2.0.1-py2.py3-none-any.whl.metadata (6.7 kB)
 Collecting mpmath<1.4,>=1.1.0 (from sympy->-r requirements.txt (line 7))
 Downloading mpmath-1.3.0-py3-none-any.whl.metadata (8.6 kB)
 Collecting pycparser (from cffi>=1.0.0->pyprosep->-r requirements.txt (line 6))
 Downloading pycparser-3.0-py3-none-any.whl.metadata (8.2 kB)
 Collecting decorator>=5.1.0 (from ipython>=7.23.1->ipykernel->-r
 requirements.txt (line 1))
 Downloading decorator-5.3.1-py3-none-any.whl.metadata (3.9 kB)
 Collecting ipython-pygments-lexers>=1.0.0 (from ipython>=7.23.1->ipykernel->-r
 requirements.txt (line 1))
 Downloading ipython_pygments_lexers-1.1.1-py3-none-any.whl.metadata (1.1 kB)
 Collecting jedi>=0.18.2 (from ipython>=7.23.1->ipykernel->-r requirements.txt
 (line 1))
 Downloading jedi-0.20.0-py2.py3-none-any.whl.metadata (23 kB)
 Collecting pexpect>4.6 (from ipython>=7.23.1->ipykernel->-r requirements.txt
 (line 1))
 Downloading pexpect-4.9.0-py2.py3-none-any.whl.metadata (2.5 kB)
 Collecting prompt_toolkit<3.1.0,>=3.0.41 (from ipython>=7.23.1->ipykernel->-r
 requirements.txt (line 1))
 Downloading prompt_toolkit-3.0.52-py3-none-any.whl.metadata (6.4 kB)
 Collecting pygments>=2.14.0 (from ipython>=7.23.1->ipykernel->-r
 requirements.txt (line 1))
 Downloading pygments-2.20.0-py3-none-any.whl.metadata (2.5 kB)
 Collecting stack_data>=0.6.0 (from ipython>=7.23.1->ipykernel->-r
 requirements.txt (line 1))
 Downloading stack_data-0.6.3-py3-none-any.whl.metadata (18 kB)
 Collecting typing_extensions>=4.6 (from ipython>=7.23.1->ipykernel->-r
 requirements.txt (line 1))
 Downloading typing_extensions-4.15.0-py3-none-any.whl.metadata (3.3 kB)
 Collecting wcwidth (from
 prompt_toolkit<3.1.0,>=3.0.41->ipython>=7.23.1->ipykernel->-r requirements.txt
 (line 1))
 Downloading wcwidth-0.7.0-py3-none-any.whl.metadata (36 kB)
 Collecting parso<0.9.0,>=0.8.6 (from
 jedi>=0.18.2->ipython>=7.23.1->ipykernel->-r requirements.txt (line 1))
 Downloading parso-0.8.7-py2.py3-none-any.whl.metadata (8.2 kB)
 Collecting platformdirs>=2.5 (from jupyter-core!=6.0.*,>=5.1->ipykernel->-r
 requirements.txt (line 1))
 Downloading platformdirs-4.10.0-py3-none-any.whl.metadata (5.5 kB)
 Collecting ptyprocess>=0.5 (from pexpect>4.6->ipython>=7.23.1->ipykernel->-r
 requirements.txt (line 1))
 Downloading ptyprocess-0.7.0-py2.py3-none-any.whl.metadata (1.3 kB)
 Collecting six>=1.5 (from python-dateutil>=2.8.2->pandas->-r requirements.txt
 (line 3))

```

    Downloading six-1.17.0-py2.py3-none-any.whl.metadata (1.7 kB)
Collecting executing>=1.2.0 (from
stack_data>=0.6.0->ipython>=7.23.1->ipykernel->-r requirements.txt (line 1))
    Downloading executing-2.2.1-py2.py3-none-any.whl.metadata (8.9 kB)
Collecting asttokens>=2.1.0 (from
stack_data>=0.6.0->ipython>=7.23.1->ipykernel->-r requirements.txt (line 1))
    Downloading asttokens-3.0.1-py3-none-any.whl.metadata (4.9 kB)
Collecting pure-eval (from stack_data>=0.6.0->ipython>=7.23.1->ipykernel->-r
requirements.txt (line 1))
    Downloading pure_eval-0.2.3-py3-none-any.whl.metadata (6.3 kB)
Downloading ipykernel-7.2.0-py3-none-any.whl (118 kB)
Downloading
numpy-2.4.6-cp311-cp311-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (16.9
MB)
                                16.9/16.9 MB
144.4 kB/s  0:02:13m0:00:0100:05
Downloading
pandas-3.0.3-cp311-cp311-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl (11.3
MB)
                                11.3/11.3 MB
122.5 kB/s  0:01:28m0:00:0300:05
Downloading
matplotlib-3.10.9-cp311-cp311-manylinux2014_x86_64.manylinux_2_17_x86_64.whl
(8.8 MB)
                                8.8/8.8 MB
110.0 kB/s  0:01:17m0:00:0200:04
Downloading
scipy-1.17.1-cp311-cp311-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (35.3
MB)
                                35.3/35.3 MB
113.1 kB/s  0:05:41m0:00:0200:09
Downloading sympy-1.14.0-py3-none-any.whl (6.3 MB)
                                6.3/6.3 MB
64.2 kB/s  0:01:08m0:00:0100:04m
Downloading mpmath-1.3.0-py3-none-any.whl (536 kB)
                                536.2/536.2 kB
78.7 kB/s  0:00:106m-:--:--
Downloading Fraction-2.2.0-py3-none-any.whl (4.7 kB)
Downloading
cffi-2.0.0-cp311-cp311-manylinux2014_x86_64.manylinux_2_17_x86_64.whl (215 kB)
Downloading comm-0.2.3-py3-none-any.whl (7.3 kB)
Downloading
contourpy-1.3.3-cp311-cp311-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (355
kB)
Downloading cycler-0.12.1-py3-none-any.whl (8.3 kB)
Downloading debugpy-1.8.20-cp311-cp311-manylinux_2_34_x86_64.whl (3.2 MB)
                                3.2/3.2 MB
23.6 kB/s  0:02:07m0:00:0200:08m

```

```

Downloading
fonttools-4.63.0-cp311-cp311-manylinux2014_x86_64.manylinux_2_17_x86_64.whl (5.1
MB)
5.1/5.1 MB
61.0 kB/s 0:01:01m0:00:0200:06m
Downloading ipython-9.14.0-py3-none-any.whl (627 kB)
627.5/627.5 kB
142.7 kB/s 0:00:04eta 0:00:01
Downloading prompt_toolkit-3.0.52-py3-none-any.whl (391 kB)
Downloading decorator-5.3.1-py3-none-any.whl (10 kB)
Downloading ipython_pygments_lexers-1.1.1-py3-none-any.whl (8.1 kB)
Downloading jedi-0.20.0-py2.py3-none-any.whl (4.9 MB)
4.9/4.9 MB
133.2 kB/s 0:00:37 eta 0:00:02
Downloading parso-0.8.7-py2.py3-none-any.whl (107 kB)
Downloading jupyter_client-8.8.0-py3-none-any.whl (107 kB)
Downloading jupyter_core-5.9.1-py3-none-any.whl (29 kB)
Downloading
kiwisolver-1.5.0-cp311-cp311-manylinux2014_x86_64.manylinux_2_17_x86_64.whl (1.4
MB)
1.4/1.4 MB
140.7 kB/s 0:00:10 eta 0:00:01
Downloading matplotlib_inline-0.2.2-py3-none-any.whl (9.5 kB)
Downloading nest_asyncio-1.6.0-py3-none-any.whl (5.2 kB)
Downloading packaging-26.2-py3-none-any.whl (100 kB)
Downloading pexpect-4.9.0-py2.py3-none-any.whl (63 kB)
Downloading
pillow-12.2.0-cp311-cp311-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (7.1
MB)
7.1/7.1 MB
84.9 kB/s 0:01:17 eta 0:00:04m
Downloading platformdirs-4.10.0-py3-none-any.whl (22 kB)
Downloading psutil-7.2.2-cp36-abi3-
manylinux2010_x86_64.manylinux_2_12_x86_64.manylinux_2_28_x86_64.whl (155 kB)
Downloading ptyprocess-0.7.0-py2.py3-none-any.whl (13 kB)
Downloading pygments-2.20.0-py3-none-any.whl (1.2 MB)
1.2/1.2 MB
131.8 kB/s 0:00:09 eta 0:00:02
Downloading pyparsing-3.3.2-py3-none-any.whl (122 kB)
Downloading python_dateutil-2.9.0.post0-py2.py3-none-any.whl (229 kB)
Downloading
pyzmq-27.1.0-cp311-cp311-manylinux_2_26_x86_64.manylinux_2_28_x86_64.whl (857
kB)
857.0/857.0 kB
77.2 kB/s 0:00:10 eta 0:00:01
Downloading six-1.17.0-py2.py3-none-any.whl (11 kB)
Downloading stack_data-0.6.3-py3-none-any.whl (24 kB)
Downloading asttokens-3.0.1-py3-none-any.whl (27 kB)

```

```

Downloading executing-2.2.1-py2.py3-none-any.whl (28 kB)
Downloading tornado-6.5.6-cp39-abi3-
manylinux1_x86_64.manylinux_2_28_x86_64.manylinux_2_5_x86_64.whl (448 kB)
Downloading traitlets-5.15.0-py3-none-any.whl (85 kB)
Downloading typing_extensions-4.15.0-py3-none-any.whl (44 kB)
Downloading attrdict-2.0.1-py2.py3-none-any.whl (9.9 kB)
Downloading pure_eval-0.2.3-py3-none-any.whl (11 kB)
Downloading pycparser-3.0-py3-none-any.whl (48 kB)
Downloading wcwidth-0.7.0-py3-none-any.whl (110 kB)
Building wheels for collected packages: pypropep
  Building wheel for pypropep (pyproject.toml) ... error
  error: subprocess-exited-with-error

```

```

× Building wheel for pypropep

```

```

(pyproject.toml) did not run successfully.

```

```

  exit code: 1

```

```

> [38 lines of output]

```

```

  running bdist_wheel
  running build
  running build_py
  creating build/lib.linux-x86_64-cpython-311
  copying cpropep_build.py -> build/lib.linux-x86_64-cpython-311
  creating build/lib.linux-x86_64-cpython-311/pypropep
  copying pypropep/propellant.py ->
build/lib.linux-x86_64-cpython-311/pypropep
  copying pypropep/performance.py ->
build/lib.linux-x86_64-cpython-311/pypropep
  copying pypropep/error.py ->
build/lib.linux-x86_64-cpython-311/pypropep
  copying pypropep/equilibrium.py ->
build/lib.linux-x86_64-cpython-311/pypropep
  copying pypropep/__init__.py ->
build/lib.linux-x86_64-cpython-311/pypropep
  creating build/lib.linux-x86_64-cpython-311/pypropep/cpropep
  copying pypropep/cpropep/__init__.py ->
build/lib.linux-x86_64-cpython-311/pypropep/cpropep
  creating build/lib.linux-x86_64-cpython-311/pypropep/data
  copying pypropep/data/thermo.dat ->
build/lib.linux-x86_64-cpython-311/pypropep/data
  copying pypropep/data/propellant.dat ->
build/lib.linux-x86_64-cpython-311/pypropep/data
  running build_ext
  generating cffi module
'build/temp.linux-x86_64-cpython-311/pypropep.cpropep._cpropep.c'
  creating build/temp.linux-x86_64-cpython-311
  building 'pypropep.cpropep._cpropep' extension
  creating

```

```

build/temp.linux-x86_64-cpython-311/build/temp.linux-x86_64-cpython-311
    creating
build/temp.linux-x86_64-cpython-311/pypropep/cpropep/libcompat/src
    creating
build/temp.linux-x86_64-cpython-311/pypropep/cpropep/libcpropep/src
    creating
build/temp.linux-x86_64-cpython-311/pypropep/cpropep/libnum/src
    creating
build/temp.linux-x86_64-cpython-311/pypropep/cpropep/libthermo/src
    /home/azid0/.local/anaconda3/envs/qchem/bin/x86_64-conda-linux-
gnu-cc -march=nocona -mtune=haswell -ftree-vectorize -fPIC -fstack-protector-
strong -fno-plt -O2 -ffunction-sections -pipe -isystem
/home/azid0/.local/anaconda3/envs/qchem/include -DNDEBUG -D_FORTIFY_SOURCE=2 -O2
-isystem /home/azid0/.local/anaconda3/envs/qchem/include -fPIC
-Ipypropep/cpropep/libnum/include/ -Ipypropep/cpropep/libthermo/include/
-Ipypropep/cpropep/libcpropep/include/ -Ipypropep/cpropep/libcompat/include/
-I/home/azid0/.local/anaconda3/envs/qchem/include/python3.11 -c
build/temp.linux-x86_64-cpython-311/pypropep.cpropep._cpropep.c -o build/temp.li
nux-x86_64-cpython-311/build/temp.linux-x86_64-cpython-
311/pypropep.cpropep._cpropep.o
    In file included from
pypropep/cpropep/libcpropep/include/equilibrium.h:11,
        from
pypropep/cpropep/libthermo/include/thermo.h:4,
        from
build/temp.linux-x86_64-cpython-311/pypropep.cpropep._cpropep.c:574:
    pypropep/cpropep/libcompat/include/compat.h:45:3: error: cannot
use keyword 'false' as enumeration constant
    45 |     false = 0,
        |     ^~~~~
    pypropep/cpropep/libcompat/include/compat.h:45:3: note: 'false'
is a keyword with '-std=c23' onwards
    pypropep/cpropep/libcompat/include/compat.h:47:3: error: expected
';', identifier or '(' before 'bool'
    47 | } bool;
        |     ^~~~
    pypropep/cpropep/libcompat/include/compat.h:47:3: warning:
useless type name in empty declaration
    error: command
'/home/azid0/.local/anaconda3/envs/qchem/bin/x86_64-conda-linux-gnu-cc' failed
with exit code 1

```

[end of output]

note: This error originates from a subprocess, and is likely not a problem with pip.

ERROR: Failed building wheel for pypropep

Failed to build pypropep

error: failed-wheel-build-for-install

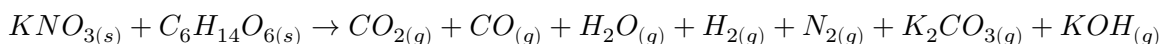
× Failed to build installable wheels for some pyproject.toml based projects

> pypropep

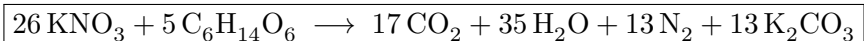
Note: you may need to restart the kernel to use updated packages.

1.2 2. A little bit on it's Stoichiometry...

We'll now solve for the following equation:



This is what annoys me with this equation. It's too spread out and at the high temperature i.e. **Adiabatic Flame Temperature** (AFT) - *that's ranges from 2100 K to 3500 K depending on the composite in question.* - one can intuitively tell that these extra products will collapse into the following:



A necessary GCD reduction step after solving the null-space problem as ensures you get the minimal positive integer coefficients that represent the equation as the free variable value will be “kinda wonky”.

This is implemented in `stoichiometric_justification.ipynb`.

1.3 3. Now the fun part...

I'll first start by importing the various libraries we need for this task.

```
[4]: %matplotlib inline
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec

# Fix deprecated collections imports
import collections.abc
import collections
if not hasattr(collections, 'MutableMapping'):
    collections.MutableMapping = collections.abc.MutableMapping
if not hasattr(collections, 'Mapping'):
    collections.Mapping = collections.abc.Mapping
if not hasattr(collections, 'Sequence'):
    collections.Sequence = collections.abc.Sequence
```



```
import pypropep as ppp
```

We'll initialize pypropep and set up our plotting as follows:

```
[5]: ppp.init()

plt.rcParams.update({
    'figure.dpi': 120,
    'axes.spines.top': False,
    'axes.spines.right': False,
    'font.family': 'DejaVu Sans'
})
```

Loaded 1921 thermo species

Loaded 1031 propellants

Then I'll define my global variables that will be used throughout the code. For this we'll need the **mechanism** which points to a yaml file that contains the details of the Grains in question and to define some lists like the **flame temp.** and **specific impulse** that will contain the calculated values from our script

We'll then parse the data in the CSV file and get the masses of KNO_3 and $C_6H_{14}O_6$ and store them as a dataframe.

```
[6]: df = pd.read_csv("../share/csv/Thermochemical_Justification.csv")

print(df.dtypes)
print(df[['Mass KNO3 (g)', 'Mass Sorbitol (g)']].head())
print(df.isna().sum())

df['Mass KNO3 (g)'] = pd.to_numeric(df['Mass KNO3 (g)'], errors='coerce')
df['Mass Sorbitol (g)'] = pd.to_numeric(df['Mass Sorbitol (g)'],
    ↪errors='coerce')
df = df.dropna(subset=['Mass KNO3 (g)', 'Mass Sorbitol (g)'])

df.head()
```

```
KN03 (%)          float64
Sorbitol (%)       float64
Mass KNO3 (g)      float64
Mass Sorbitol (g)  float64
C*                 float64
Density            float64
Chamber Cp/Cv      float64
Chamber Temp       float64
Isp*               float64
Molecular Weight   float64
dtype: object
   Mass KNO3 (g)  Mass Sorbitol (g)
```

0	726.80	986.83
1	730.43	981.90
2	734.07	976.96
3	737.70	972.03
4	741.34	967.09

KN03 (%)	0
Sorbitol (%)	0
Mass KN03 (g)	0
Mass Sorbitol (g)	0
C*	161
Density	161
Chamber Cp/Cv	161
Chamber Temp	161
Isp*	161
Molecular Weight	161

dtype: int64

```
[6]:
```

	KN03 (%)	Sorbitol (%)	Mass KN03 (g)	Mass Sorbitol (g)	C*	Density \
0	50.00	50.00	726.80	986.83	NaN	NaN
1	50.25	49.75	730.43	981.90	NaN	NaN
2	50.50	49.50	734.07	976.96	NaN	NaN
3	50.75	49.25	737.70	972.03	NaN	NaN
4	51.00	49.00	741.34	967.09	NaN	NaN

	Chamber Cp/Cv	Chamber Temp	Isp*	Molecular Weight
0	NaN	NaN	NaN	NaN
1	NaN	NaN	NaN	NaN
2	NaN	NaN	NaN	NaN
3	NaN	NaN	NaN	NaN
4	NaN	NaN	NaN	NaN

Now that's sorted, we can calculate the following data as per the csv file:

1. Density (ρ) We'll be using the following formula:

$$\rho_p = \frac{1}{\frac{f_0}{\rho_0} + \frac{f_1}{\rho_1}}$$

where f_n is the **mass fraction** of the propellant component and ρ_n is the respective density.

2. Chamber c_p/c_v (k or γ) This is the **specific heat ratio** and it is used to determine the chamber pressure. It's given by:

$$k = \frac{1}{1 - \frac{X}{1-X} \frac{R}{C_s + C_p}}$$

where R is the **universal gas constant**, $\frac{X}{1-X}$ is the **mole fraction of condensed phase products**, C_s is the **specific heat of the mixture of condensed phase products** and C_p is the **specific heat of the mixture of gaseous products**.

It should be noted that this is for a **2-phase flow** where we have both condensed(*solid*) and gaseous phase products from this combustion.(Ikiara, 2025)

3. Chamber velocity (C^*) This is essential in determining the speed at which the propellant will burn at. It's given by:

$$C^* = \sqrt{\frac{R T_o}{k \left(\frac{2}{k+1}\right)^{\frac{k+1}{k-1}}}}$$

where T_o is the **chamber temperature**.

4. Specific impulse (I_{sp}) This is the crucial factor that determines the thrust of our SRM. It is given by:

$$I_{sp} = \frac{1}{g} \sqrt{2T_o \left(\frac{R}{M}\right) \left(\frac{k}{k-1}\right) \left[1 - \frac{P_e}{P_o}\right]^{\frac{k-1}{k}}}$$

where g is the **accelearation due to gravity**, M is the **molecular mass of the KNSB grain**, P_e and P_o are the **exit and chamber pressures**.

```
[ ]: # Components
sorb = ppp.PROPELLANTS['SORBITOL']
kno3 = ppp.PROPELLANTS['POTASSIUM NITRATE']

# Component densities (g/cm³)
RHO_KNO3      = 2.109
RHO_SORBITOL  = 1.489

# Motor operating conditions
P_CHAMBER_ATM = 68.0    # ~1000 psi, typical KNSB
P_EXIT_ATM    = 1.0     # sea level

# Accounting for the two-phase flow as (Nakka, 2025)

# Simulation loop
isps, chamber_temps, c_stars, isexs = [], [], [], []
densities, cps, cvs, gammas, of_ratios = [], [], [], [], []
```

From this data we can show the relationship between I_{sp} and mass of KNO_3 used:

```
[ ]: # SORB_PURITY    = 0.7365
# MIN_MASS_G        = 10.0
```

```

# G0          = 9.80665

# def _fill_nan():
#     for lst in [isps, c_stars, chamber_temps, gammas, cps, cus, of_ratios,
# densities]:
#         lst.append(np.nan)

# for i, row in df.iterrows():

#     m_fu          = m_fu_liquid * SORB_PURITY

#     if m_ox < MIN_MASS_G or m_fu < MIN_MASS_G:
#         print(f"[SKIP] KNO3={kno3_pct:.2f}% - mass too low for solver
# (m_ox={m_ox:.1f}g, m_fu={m_fu:.1f}g)")
#         _fill_nan()      # see helper below
#         continue

#     of_ratio = m_ox / m_fu if m_fu != 0 else 0
#     of_ratios.append(round(of_ratio, 4))

for _, row in df.iterrows():
    pct_ox = row['KNO3 (%)']
    pct_fu = row['Sorbitol (%)']
    m_ox    = row['Mass KNO3 (g)']
    m_fu    = row['Mass Sorbitol (g)']

    print(f"[INFO] Siulating KNO3={kno3_pct}%")

    try:
        p = ppp.FrozenPerformance()
        p.add_propellants_by_mass([(sorb, m_fu), (kno3, m_ox)])
        p.set_state(P=P_CHAMBER_ATM, Pe=P_EXIT_ATM)

        # isps.append(round(p.performance.Isp, 4))
        chamber_temps.append(round(p.properties[0].T, 4))
#         c_stars.append(round(e.performance.cstar, 4))
#         cps.append(round(e.properties[0].Cp, 4))
#         cus.append(round(e.properties[0].Cu, 4))
#         isexs.append(round(e.properties[0].Isex, 4))
#         gammas.append(round(e.properties[0].Cp / e.properties[0].Cu, 4))
    except Exception as e:
        print(f"[FAIL] KNO3={pct_ox:.2f}% Sorbitol={pct_fu:.2f}% - solver error:
        {e}")
        continue

```

```

f_ox = pct_ox / 100.0
f_fu = pct_fu / 100.0
rho_p = 1.0 / (f_ox / RHO_KNO3 + f_fu / RHO_SORBITOL)
densities.append(rho_p)

print(f"[INFO] Simulation completed for {len(isps)} cases.")

```

```

LU: matrix is singular, no unique solution.
LU: matrix is singular, no unique solution.
LU: matrix is singular, no unique solution.
LU: matrix is singular, no unique solution.
LU: matrix is singular, no unique solution.
LU: matrix is singular, no unique solution.
LU: matrix is singular, no unique solution.
LU: matrix is singular, no unique solution.

```

The Kernel crashed while executing code in the current cell or a previous cell.

Please review the code in the cell(s) to identify a possible cause of the failure.

Click [here](https://aka.ms/vscodeJupyterKernelCrash) for more info.

View Jupyter [log](command:jupyter.viewOutput) for further details.

We'll then write the results back to the DataFrame and display it for confirmation:

```

[ ]: n = min(len(df), len(c_stars), len(densities), len(cps), len(cvs),
           len(chamber_temps), len(isps), len(of_ratios))
df = df.iloc[:n].copy()

df['C*'] = c_stars[:n]
df['Density'] = densities[:n]
df['Chamber Cp'] = cps[:n]
df['Chamber Cv'] = cvs[:n]
df['Chamber Temp'] = chamber_temps[:n]
df['Isp*'] = isps[:n]
df['Chamber Cp/Cv'] = gammas[:n]
df['Isex'] = isexs[:n]
df['O/F'] = of_ratios[:n]

df.to_csv("./share/csv/Thermochemical_Justification.csv", index=False)
df.head()

```

We can also outline the relationship between the flame temperatures and mass of KNO_3 used:

```
[ ]: import os

kno3_pct = df['KNO3 (%)']
sweet_spot_idx = df[df['KNO3 (%)'] == 65.0].index[0]

fig = plt.figure(figsize=(15, 4))
gs = gridspec.GridSpec(1, 3, figure=fig, wspace=0.35)

panels = [
    (gs[0], kno3_pct, isps, 'Specific Impulse (s)', 'Isp vs KNO3 Loading'),
    (gs[1], kno3_pct, chamber_temps, 'Chamber Temperature (K)', 'Tchamber vs KNO3 Loading'),
    (gs[2], kno3_pct, c_stars, 'Characteristic Velocity C* (m/s)', 'C* vs KNO3 Loading'),
]

for spec, x, y, ylabel, title in panels:
    ax = fig.add_subplot(spec)
    ax.plot(x, y, 'o-', lw=1.5, ms=4, color='steelblue')
    ax.axvline(x=kno3_pct[sweet_spot_idx], color='tomato',
               linestyle='--', lw=1.2, label=f"Optimal: {kno3_pct[sweet_spot_idx]:.2f}%")
    ax.set_xlabel('KNO3 Loading (%)')
    ax.set_ylabel(ylabel)
    ax.set_title(title, fontsize=10)
    ax.legend(fontsize=8)

plt.suptitle('KNSB Propellant Performance Sweep', fontweight='bold')

# Create directories if they don't exist
output_dir = './share/images'
os.makedirs(output_dir, exist_ok=True)

# Save figure
plt.savefig(os.path.join(output_dir, 'knsb_performance_sweep.png'),
            bbox_inches='tight')
plt.show()
```

```
[ ]: import os

opt_target = 1.857
opt_idx = df['O/F'].sub(opt_target).abs().idxmin()
opt_of = df.loc[opt_idx, 'O/F']

fig = plt.figure(figsize=(15, 4))
gs = gridspec.GridSpec(1, 3, figure=fig, wspace=0.35)
```

```

panels = [
    (gs[0], isps, 'Specific Impulse (s)', 'Isp'),
    (gs[1], chamber_temps, 'Chamber Temperature (K)', 'T_c'),
    (gs[2], c_stars, 'Characteristic Velocity (m/s)', 'C*'),
]

for spec, y, ylabel, label in panels:
    ax = fig.add_subplot(spec)
    ax.plot(of_ratios, y, 'o-', lw=1.5, ms=4, color='steelblue', label=label)
    ax.axvline(opt_of, color='tomato', ls='--', lw=1.2,
               label=f'Optimum O/F = {opt_of:.3f}')
    ax.set_xlabel('O/F Ratio (KNO / Sorbitol)')
    ax.set_ylabel(ylabel)
    ax.legend(fontsize=8)

plt.suptitle('KNSB Performance vs. O/F Ratio', fontweight='bold')
plt.tight_layout()

# Create directories if they don't exist
output_dir = './share/images'
os.makedirs(output_dir, exist_ok=True)

# Save figure
plt.savefig(os.path.join(output_dir, 'knsb_performance_of_ratio.png'),
            bbox_inches='tight')
plt.show()

```

1.4 4. What we can learn from this data

We'll find that with more mass of KNO_3 used, do these critical factors i.e. I_{sp} , chamber temperature and C^* increase. In reality, KNSB grains made with such concentrations of KNO_3 are harder to process and for the motor casing to handle.

We'll also find that with less mass of KNO_3 used, the motor is cooler but less energetic. We can see that at 65% : 35% i.e. when mass of KNO_3 used is 944.84g, we have the “sweet spot”.

It's also worth mentioning that when too much oxidizer is used, the KNSB grain becomes brittle and sensitive to moisture while when too much fuel is used, incomplete combustion and lower I_{sp} is noted. (Nakka, 2025)

Since it's a 2-phase flow analysis: ### 1. The gaseous phase The dominant species from the combustion tell us the following in the table below:

Species	Mole fraction	What it means
H_2O	0.304	Primary hydrogen combustion product
CO	0.185	Incomplete carbon oxidation
H_2	0.176	Unburned hydrogen
CO_2	0.122	Complete carbon oxidation
N_2	0.106	Nitrogen release from KNO_3

```
[ ]: # Quick sanity check
# After the loop, run this for the optimal ratio point
p_opt = ppp.FrozenPerformance()
p_opt.add_propellants_by_mass([
    (kno3, df.loc[opt_idx, 'Mass KNO3 (g)']),
    (sorb, df.loc[opt_idx, 'Mass Sorbitol (g)']),
])
p_opt.set_state(P=68., Pe=1.0)

print("=== Optimal KNSB (O/F = {:.3f}) ===".format(opt_of))
print("\nGaseous chamber products:")
import pprint
pprint.pprint(p_opt.composition['chamber'][:8])
print("\nCondensed chamber products (K CO should appear here):")
pprint.pprint(p_opt.composition_condensed['chamber'])
```

1.5 5. What's in-store for the future of this simulation

We plan on performing a CFD Simulation of 1-D isentropic nozzle flow i.e.:

$$M_e = f(P_c, P_e, \gamma), v_e = \sqrt{\frac{2\gamma}{\gamma-1} \frac{RT_e}{M_w} \left[1 - \left(\frac{P_e}{P_c} \right)^{\frac{\gamma-1}{\gamma}} \right]}$$

using OpenFOAM and ANSYS programs hence the 2 folders in the **/share** directory. This is pretty taxing so far and will require some time.

1.6 6. In conclusion...

We justified the optimal KNSB ratio i.e. 65% : 35%, as it provides a high specific impulse with efficient combustion, *indicated by I_{sp} and C^** , while maintaining safe thermal characteristics and physical integrity. It is a well-characterized formulation that makes it a reliable and practical choice for our SRM.

1.7 References

- [1] R. Nakka, "KNSB Propellant," Richard Nakka's Experimental Rocketry Web Site, 2025. [Online]. Available: <https://www.nakka-rocketry.net/sorb.html>
- [2] R. Nakka, "Solid Rocket Motor Theory - Two-phase flow," Richard Nakka's Experimental Rocketry Web Site, 2025. [Online]. Available: https://www.nakka-rocketry.net/th_2phf.htm
- [3] R. Nakka, "Solid Rocket Motor Theory – Impulse and C-star," Richard Nakka's Experimental Rocketry Web Site, 2025. [Online]. Available: https://www.nakka-rocketry.net/th_imp.html
- [4] R. Nakka, "Two-phase flow theory," Richard Nakka's Experimental Rocketry Web Site, Oct. 28, 2011. [Online]. Available: [nakka-rocketry.net](https://www.nakka-rocketry.net). [Accessed: May 31, 2026].

- [5] R. Nakka, “Potassium nitrate/sorbitol (KNSB) propellant performance characteristics,” Richard Nakka’s Experimental Rocketry Web Site, Mar. 22, 2011. [Online]. Available: nakka-rocketry.net. [Accessed: May 31, 2026].
- [5] J. Bonnie, J. Zehe, and S. Gordon, “NASA Glenn Coefficients for Calculating Thermodynamic Properties of Individual Species,” NASA/TP—2002-211556, Glenn Research Center, Cleveland, 2002.
- [6] T. McReary, Experimental Composite Propellant: An Introduction To Properties And Preparation OF Composite Propellants: Design, Construction, Testing, and Characteristics Of Small Rocket Motors, 1st ed. 2020.
- [7] D. Mazza and E. Canuto, Fundamental Chemistry with MATLAB. Oxford Publishing, 2022.